

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

Organisation of Islamic Cooperation (OIC)

Board Bazar, Gazipur, Bangladesh

Department of Computer Science & Engineering (CSE)

SOFTWARE DESIGN REPORT

KIDSCRIBE

A Tablet Application that Teaches Children Aged 5 to 8 to Write by Hand

Course Title: Software Project Lab-II

Course Code: SWE 4404

Academic Year: 2024-25

Group 18

Hasan Mahi — 230042129

Zuhaaer Mustafa Sadab — 230042132

Md Shahib Al Kabir — 230042157

Supervised by

Farzana Tabassum

Department of Computer Science & Engineering

Submission: 1 November 2026

1. Purpose and Scope

This document describes how KidScribe is built. The Software Requirements Specification says *what* the application must do; this report says *how* the code is arranged to do it, and why each arrangement was chosen over the alternatives we considered. It covers the architecture, the way the parts talk to each other, the classes and data, the algorithms that mark a child's handwriting, the screen flow, and the trade-offs.

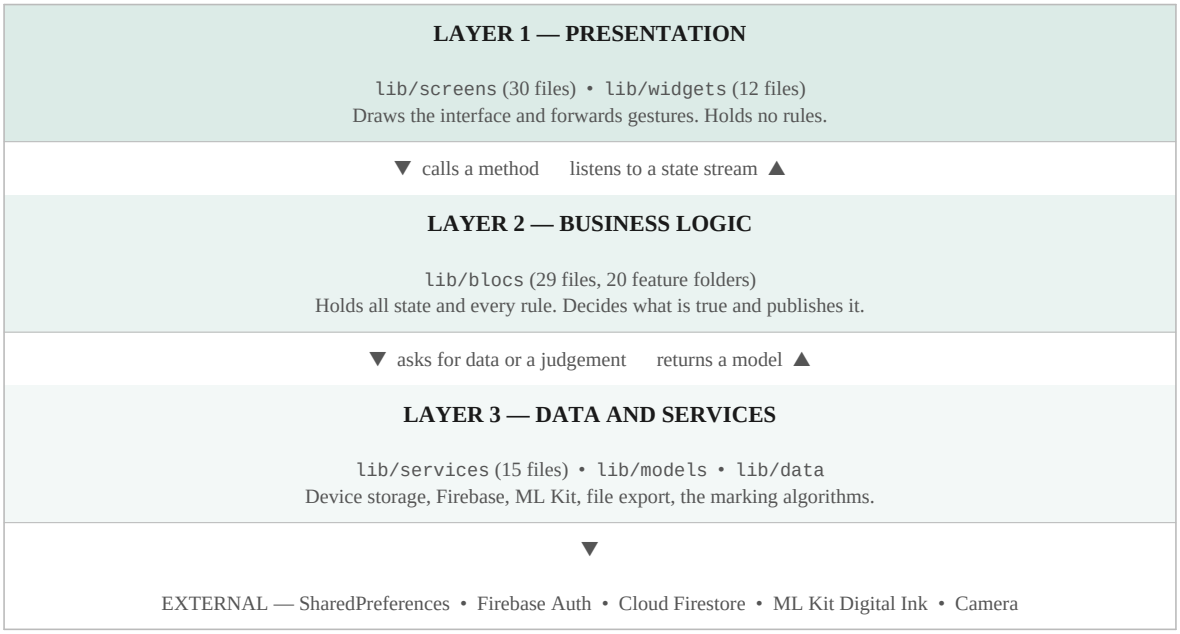
KidScribe is an offline Android tablet application that teaches children aged five to eight to write by hand. A child writes on the screen with a stylus; the application marks the writing immediately and says what to fix. Parents and teachers have their own side of the application for profiles, classes, tasks and reports. Children do not have accounts.

Every design decision in this document serves at least one requirement in the SRS, and Section 10 maps them.

2. Architecture

2.1 The Three Layers

KidScribe uses a three-layer architecture implemented with the BLoC pattern (Business Logic Component). The rule that defines it is that **dependencies point downwards only**.



A screen never calls a service directly, and no service imports anything from a screen. That one rule is the whole point of the architecture: it means the rules can be tested without rendering anything, which is why 58 of the 80 unit tests in the SRS run on a desktop with no emulator attached.

2.2 Bloc or Cubit

`flutter_bloc` offers two forms. A **Bloc** receives named event objects; a **Cubit** is the same thing driven by ordinary method calls. We use a full Bloc in exactly one place — authentication — because sign-in has distinct, auditable events (`LoginRequested`, `SignupRequested`, `LogoutRequested`) that are worth naming. Everywhere else a Cubit carries the same guarantees with far less code, so the other 19 feature folders are Cubits.

2.3 The One Documented Exception

Pen input is deliberately kept out of the state layer. A stylus produces roughly 60 points per second; routing each one through a bloc would rebuild the widget tree 60 times a second for no benefit. `WritingPad` therefore accumulates points in a local controller and hands the finished strokes to the bloc only when the child presses Check. This is a conscious exception to the layering rule, recorded here and in `ARCHITECTURE.md` so that it is not mistaken for an oversight.

2.4 One-Shot Effects Versus State

A redraw and an event are different things, and conflating them is the classic bug in this architecture: a dialogue that reopens every time the screen rebuilds. Our solution is a **tick counter**. Each state that can trigger a one-shot effect carries an integer — `resultTick`, `guidedTick`, `celebrateTick` — which the bloc increments when the event happens. A `BlocListener` with `listenWhen: (a, b) => a.tick != b.tick` fires exactly once per increment and never on an ordinary redraw.

3. Package Organisation

Folders correspond to layers rather than to features, so the path of a file states what it is permitted to touch.

```
lib/
|-- main.dart              entry point; builds the service objects once and
|                          provides them to the whole widget tree
|-- firebase_options.dart  generated Firebase configuration
|-- theme/          2      colours, text styles, spacing
|-- models/         4      plain data classes, no logic, no imports from the app
|-- data/           3      letter templates, word list, sentence list
|-- services/      15      LAYER 3: storage, Firebase, ML Kit, marking, export
|-- blocs/         29      LAYER 2: one folder per feature, 20 features
|   |-- auth/              the only full Bloc: events, states, bloc
|   |-- letter/            guided and free letter practice
|   |-- sentence/          Stage 4 card placement and checking
|   +-- ...                badges, games, classes, assignments, premium, sync
|-- widgets/       12      LAYER 1 shared: KidPage, KidCard, KidButton,
|                          WritingPad, mascot, park scene
+-- screens/       30      LAYER 1: one file per screen
    |-- learn/             the five stages
    |-- games/             Letter Match, Letter Ninja, Free Writing
    +-- assign/            giving tasks, grading, written feedback
```

Why `models` and `data` import nothing. These two folders are pure: no Flutter widgets, no Firebase, no plugins. That is what lets a unit test construct a `TraceItem` or read the real letter templates without a device.

4. Class Design

4.1 How Class Boundaries Were Decided

- **Single responsibility.** One class answers one question. `TraceEvaluator` answers 'is this traced well?'; `ProgressService` answers 'what has this child done?'. Neither knows about the other.
- **Layering.** A class may call only the layer below it.
- **Reuse.** Anything needed in two places becomes its own class. `PasswordRules` exists because the sign-up screen was writing the same rules twice.
- **Testability by construction.** Every bloc takes its services as constructor arguments, so a test can pass its own. No bloc creates a service inside itself.

4.2 Models (Layer 3, Pure Data)

All models are immutable and extend `Equatable`, so two instances with the same contents are equal. This is what makes `BlocBuilder` able to skip a redraw when nothing changed.

Class	Fields	Serves
`UserModel`	email, password, name, role	FR-1, FR-2
`ChildProfile`	id, name, age	FR-3
`SchoolClass`	id, name, students	FR-4
`TraceItem`	id, glyph, name, hint, strokes (normalised)	FR-5, FR-6
`TraceResult`	score, stars, passed, coverage, precision, orderTip	FR-7
`FreehandResult`	percent, passed, recognizedAs, usedMl, kidMessage, grownUpReport	FR-6, FR-7
`PracticeSentence`	id, words, emoji	FR-9
`Assignment`	id, title, kind, items, inOrder, dueAtMs, testMinutes, doneKeys, grade	FR-17, FR-18, FR-23, FR-24
`FeedbackNote`	id, target, text, fromRole, atMs	FR-22
`ScopeSummary`	strokesDone, lettersDone, wordsDone, totalChecks, lastPractice	FR-19
`BadgeDef`	id, title, emoji, description	FR-14

4.3 Services (Layer 3, Behaviour)

Service	Responsibility	Depends on
`TraceEvaluator`	Marks a guided trace: coverage, precision, stars, stroke order	nothing
`FreehandChecker`	Marks unguided writing: ML recognition blended with geometry	ML Kit
`SentenceChecker`	Checks a built sentence: full stop, word order, capitals	nothing
`ProgressService`	Reads and writes every learner's progress under a scope	SharedPreferences
`BadgeService`	Computes earned badges and the daily streak from progress	ProgressService
`FamilyService`	Child profiles, classes, students, the active learner	SharedPreferences
`AssignmentService`	Tasks, deadlines, timed tests, grading, feedback notes	SharedPreferences
`SyncService`	Uploads and merges progress with Firestore	Firestore, Auth
`PremiumService`	Which features are locked; demo unlock	SharedPreferences
`ReportService`	Builds and exports a progress report file	ProgressService
`ToyWordService`	Turns a photographed toy into a practice word	ML Kit, camera
`ReminderService`	Schedules the daily practice notification	local notifications
`PasswordRules`	The single definition of a valid password	nothing
`current_user.dart`	Returns the signed-in uid, or a guest id if Firebase is absent	Auth (guarded)

The four services that depend on nothing are the ones holding the interesting logic, and that is not a coincidence: a service with no dependencies can be tested by calling it.

5. Data Design

5.1 Why There Is No Database

The whole dataset for one learner is a handful of small documents that are always read in full: which strokes are done, which letters have which scores, which words and sentences are finished, which days were practised. There are no joins and no queries. A relational database would add a schema, migrations, and a native plugin for no benefit at this size, so storage is `SharedPreferences`, the Android key-value store, with each document encoded as a JSON string.

5.2 The Scope Rule

A single tablet is shared by a whole class, so every key carries a **scope** that says whose progress it is. The rule has a strict precedence, implemented in one function so that every consumer inherits it:

```
scope =  teacher's currently selected student      (classroom mode)
    else parent's currently active child           (family mode)
    else the signed-in account's uid               (grown-up practising)
    else 'guest'                                   (Firebase unavailable)
```

Because the rule lives in one place, adding classroom mode later required no change in badges, reports, the class dashboard, or synchronisation — they all read through `ProgressService` and were already correct.

5.3 Storage Schema

Key	Value	Example
`trace_progress_<scope>`	stroke id → stars	`{"s1_standing": 3}`
`letters_v2_<scope>`	letter → {guided stars, free percent}	`{"A": {"guided": 3, "free": 82}}`
`words_v1_<scope>`	word → done	`{"CAT": 1}`
`sentences_v1_<scope>`	sentence id → done	`{"s01": 1}`
`games_v1_<scope>`	game → best result	`{"match_best": 4}`
`practice_log_<scope>`	date → number of checks	`{"2026-10-28": 12}`
`badges_<scope>`	list of badge ids already shown	`["first_letter", "streak_3"]`
`assignments_<scope>`	list of tasks and timed tests	`[{id, kind, items, dueAtMs}]`
`feedback_<scope>`	list of written notes	`[{id, target, text, fromRole}]`

The version suffix matters. `letters_v2` is named that way because the shape of the value changed when free-hand practice was added. An old installation's `letters_v1` is simply ignored rather than crashing the decoder, which is the cheapest possible migration strategy for a key-value store.

5.4 Cloud Synchronisation

Synchronisation is optional, off by default, and writes to one Firestore document per account: `kidscribe_sync/{uid}`. Each value is stored with a type tag, `{'t': 's'|'i'|'b'|'d'|'l', 'v': value}`, because SharedPreferences distinguishes a string from an integer and a round trip through a document store would otherwise return a number as text.

The merge rule is deliberately conservative. For a number the larger value wins; for a set or list the union wins; for a map the per-key merge is applied recursively. Nothing is ever replaced wholesale. The reason is that the worst failure this product can have is a child losing their stars, and a merge that only ever moves progress upwards cannot cause it.

The accepted cost is that deletions do not propagate: a child profile deleted on one device reappears after the next sync, because the merge has no way to tell a deletion from an absence. Fixing this requires deletion tombstones carrying timestamps. We judged that addition too risky to make close to the deadline and recorded the limitation instead.

Access is controlled by a Firestore security rule restricting each document to its owner:

```
match /kidscribe_sync/{uid} {
  allow read, write: if request.auth != null && request.auth.uid == uid;
}
```

6. Algorithm Design

6.1 Normalised Coordinates

Every shape in the application — the 26 letter templates, the practice strokes, the child's own ink — is stored as fractions of a unit square between 0 and 1, never in pixels. A point is multiplied by the canvas side when it is drawn and divided by the canvas side when it is read from the pen, and those two multiplications are the only places where pixels exist.

Three things follow. The same template data is correct on a phone and on a tablet. The tolerance used when marking is a fraction of the canvas rather than a fixed pixel count, so marking is equally strict on every device. And a test can construct a stroke as plain numbers with no screen involved.

6.2 Marking a Guided Trace: `TraceEvaluator.evaluate`

The child traces over a visible template. Two independent quantities are measured, each using a distance tolerance of 0.09 of the canvas:

- **Coverage** — the fraction of the template's points that have some of the child's ink near them. Low coverage means the letter was not finished.
- **Precision** — the fraction of the child's ink that landed near the template. Low precision means scribbling or a wandering line.

The score combines them as $\texttt{coverage} \times \texttt{precision}^{0.7}$. The exponent is the design decision: it weights coverage more heavily than precision, because for a five-year-old an unfinished letter is a worse error than a slightly wobbly one. Stars are awarded at 85, 70 and 55.

6.3 Checking How It Was Written: `checkOrder`

This is what separates KidScribe from a tracing toy. Shape alone cannot tell whether a letter was written the way it is taught, so stroke order and direction are checked separately, and a failure here produces a tip rather than silently passing.

Each of the child's strokes and each template stroke is **resampled to 20 points spaced evenly along its length**. This is the step that makes the comparison possible: two strokes drawn at different speeds have different numbers of raw points, but after resampling they are directly comparable point for point. The distance between two strokes is then the average distance between corresponding points — which, crucially, is large when the same line is drawn backwards, so direction is detected rather than ignored.

Each child stroke is matched to its nearest unused template stroke within a tolerance of 0.13. The result is one of three diagnoses, each with its own message: the strokes were made in the wrong sequence, a stroke was drawn in the wrong direction, or the expected number of strokes was not made. An order failure caps the score just below the pass mark, so the shape cannot carry a letter that was written incorrectly.

6.4 Marking Unguided Writing: `FreehandChecker`

In the unguided half of Stage 2 there is no template on screen, so geometry alone is not enough — a child may write a perfectly good 'A' anywhere on the canvas at any size. The design has three parts.

- **Fit to template.** The child's ink is translated and scaled so that its bounding box matches the template's, which removes position and size from the comparison and leaves only shape. The raw, unfitted ink is what goes to the recogniser, because the model reads real stroke sequences and fitting would distort them.
- **Machine learning.** The raw strokes, with timing, are sent to ML Kit Digital Ink Recognition, which runs on the device and returns its best guesses for the character.
- **Zone coverage.** The canvas is divided into top, middle and bottom thirds, and each zone must contain ink in roughly the proportion the letter requires. This catches a letter that is the right shape but written far too small.

The final percentage blends the model's judgement and the geometric score **55 to 45**. The blend is deliberate: if the recognition model is unavailable, the geometric half still produces a usable mark, so the application degrades rather than failing.

6.5 Checking a Sentence: `SentenceChecker`

Stage 4 asks the child to build a sentence from word cards. The checker applies four rules **in a fixed order and reports only the first failure**, because a five-year-old told three things at once fixes none of them:

Order	Rule	Message the child sees
1	Every box must be filled	Fill all the boxes first.
2	The last card must be the full stop	A sentence ends with a full stop.
3	The words must be in the right order (ignoring capitals)	These words are in the wrong place.
4	Only the first word is capitalised	The first word needs a big letter.

6.6 Stage Unlocking

The five stages form a small state machine: the current stage is the first one not yet complete, and every later stage is locked. Completion is derived from stored progress rather than stored as its own flag, so the two can never disagree — there is no state to corrupt, only a function over the progress that already exists.

7. Interface Design

7.1 Screen Map

```
Splash
 +-- Login / Sign up          (parent or teacher; children never log in)
      +-- Home                shows the learner, the stage, the streak
           |-- Stage 1  Strokes          guided practice, stars
           |-- Stage 2  Letters          26 letters; guided half + free half
           |-- Stage 3  Words            whole words, letter by letter
           |-- Stage 4  Sentences        word cards, capitals, full stop
           |-- Stage 5  Grown-up hub
           |-- Games    Letter Match | Letter Ninja | Free Writing
           |-- Rewards  badges, daily streak
           |-- Parent   children, weekly report, reminder, challenges,
           |            toy-photo words, premium (demo), sync
           +-- Teacher  classes, students, needs-help view, classroom mode,
                        homework, timed tests, grading, feedback, reports
```

7.2 Design Rules for Young Children

- **Large targets.** Nothing a child must hit is smaller than 48dp.
- **Pictures before words.** A five-year-old who cannot yet write also cannot yet read fluently, so navigation uses emoji and shape, with text as a secondary cue.
- **One instruction at a time.** Feedback names a single thing to fix, never a list.
- **Two audiences, two messages.** Every result produces a `kidMessage` written to be read aloud to a child and a separate `grownUpReport` with the actual numbers. The same check serves both without either being patronising or incomprehensible.
- **Palm rejection.** Once a stylus event is seen, finger events are ignored for the rest of the session, so a hand resting on the tablet does not draw.
- **Nothing destructive within reach.** Deleting a child or a class lives on the grown-up side only.

8. Design Alternatives and Trade-Offs

Decision	Chosen	Rejected	Reasoning and accepted cost
State management	BLoC (flutter_bloc)	setState inside screens; Provider; Riverpod	setState was our first implementation and put rules inside widgets, so nothing could be tested without rendering. *Cost:* 29 bloc files and more boilerplate, accepted in exchange for a testable core.
Recognition	On-device ML Kit Digital Ink	A cloud recognition API; training our own model	A cloud call is more accurate but needs connectivity and costs per call, contradicting the offline requirement. Training our own was beyond the time available. *Cost:* a 20 MB one-off download and slightly lower accuracy, mitigated by the 55/45 blend.
Local storage	SharedPreferences + JSON	SQLite (sqflite); Hive; Isar	Small documents read whole, no queries. *Cost:* comparing across many children means loading each one, which is fine for a class of forty but would not scale to a school.
Sync conflict rule	Merge: max for numbers, union for sets	Last-write-wins replacement	Losing progress is the worst possible failure. *Cost:* deletions do not propagate.
Stroke comparison	Resample to 20 points, compare in order	Compare raw points; Hausdorff distance; DTW	Raw points are unevenly spaced and vary with speed; Hausdorff ignores order entirely, which is exactly what we need to detect. *Cost:* fine detail below the resampling interval is invisible, which at this age is not worth marking.
Child accounts	None; a grown-up selects the learner	A simple PIN or avatar login per child	Keeps credentials away from young children and makes a shared classroom tablet work. *Cost:* a child cannot switch themselves, which in a classroom is intended.

Decision	Chosen	Rejected	Reasoning and accepted cost
Payments	Demo unlock, labelled as such	Google Play Billing	Real billing needs a developer account and store review. *Cost:* the feature is a demonstration, which the interface states plainly.

9. Design for the Non-Functional Requirements

NFR	Requirement	Design decision that delivers it
NFR-01	A result within about 5 seconds	Marking is local arithmetic over at most a few hundred points, measured well under a second. The ML path adds roughly a second on device and is the only part that is not instant.
NFR-02	Works without internet	Templates, scoring, storage and recognition are all local. Firebase is used only for optional sign-in and optional sync, and the application is fully usable with neither.
NFR-03	Large buttons, few words	A shared widget set enforces the minimum target size and the text style in one place, so no screen can drift from it.
NFR-04	Learners kept separate	The scope rule, implemented once, is applied to every storage key.
NFR-05	Progress is never lost	Writes keep the better of the new and stored score; sync merges upwards only.
NFR-06	Children's data is protected	No child accounts, no analytics, no advertising, and handwriting never leaves the device.

10. Traceability from Design to Requirements

Requirement	Design element
FR-1, FR-2 — accounts and login	`AuthBloc` (the one full Bloc), Firebase Authentication, `PasswordRules`
FR-3, FR-4 — profiles and classes	`FamilyService`, `ChildProfile`, `SchoolClass`, the scope rule
FR-5, FR-6 — strokes and letters	`TraceItem` templates in normalised coordinates, `WritingPad`
FR-7 — instant feedback	`TraceEvaluator`, `checkOrder`, `FreehandChecker`, the tick-counter effect pattern
FR-8, FR-9 — words and sentences	`word_list`, `sentence_list`, `SentenceCubit`, `SentenceChecker`
FR-10 — five stages in order	Stage completion derived from progress; later stages locked
FR-11 to FR-13 — games	`blocs/games`, `screens/games`, best results in `games_v1`
FR-14, FR-15 — badges and reminders	`BadgeService` over the practice log, `ReminderService`
FR-16, FR-17 — toy words, challenges	`ToyWordService` (image labelling plus a blur check), `AssignmentService`
FR-18, FR-19 — class view, reports	Class dashboard reading per-student scopes, `ReportService`, `ScopeSummary`
FR-20 — offline storage and sync	SharedPreferences schema, `SyncService` with the upward merge
FR-21 — premium	`PremiumService` gates; demo unlock, no payment path
FR-22 to FR-24 — feedback, time limits, timed tests	`FeedbackNote`, `dueAtMs` and `testMinutes` on `Assignment`, a test mode that hides all on-screen help and submits itself

11. Design Session Feedback and the Changes Made

Feedback received	Change made
The design did not follow a recognised state-management pattern; logic was held inside widgets.	The whole application was migrated to BLoC: 20 feature folders, one full Bloc for authentication and Cubits elsewhere. Sections 2.1 to 2.3 of this document are the result.
The documents used language that was too technical for the intended audience.	This report and all user-facing text were rewritten in plain language, with each technical term explained where it first appears.
Rapid pen input routed through the state layer would be a performance problem.	Pen points are kept in a local controller and handed to the bloc only on Check. The exception is documented in Section 2.3 rather than left implicit.